

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. **Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

(10 Marks)

2. **Design and implement an Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

(10 Marks)

3. **Develop a Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

(10 Marks)

4. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.
(10 Marks)
5. **Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.
(10 Marks)
6. **Design and develop a Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.
(10 Marks)
7. **Develop a Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.
(10 Marks)
8. **Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.
(10 Marks)
9. **Develop a Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.
(10 Marks)
10. **Design, implement, and evaluate a Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.
(10 Marks)

Code of Conduct:

*I T.Chaitra(192425217) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: T.chaitra

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Intelligent Traffic Signal Control — REINFORCE vs A2C

Main idea: The RL agent observes traffic conditions and chooses which signal phase to activate. The objective is to reduce waiting time and increase traffic throughput.

State: Number of vehicles waiting at each direction.

Actions:

0 → North-South green
1 → East-West green

Reward: Negative waiting time + reward for vehicles passing through.

REINFORCE: Learns the policy directly using complete episode returns.

A2C: Uses an **Actor** to select actions and a **Critic** to estimate the value of the current state.

A2C generally has lower variance and more stable learning than REINFORCE.

CODE:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

# Traffic environment
class TrafficEnv:
    def __init__(self):
        self.state = np.array([5.0, 5.0])

    def reset(self):
        self.state = np.random.randint(1, 10, 2).astype(float)
        return self.state

    def step(self, action):
        if action == 0:
            passed = min(self.state[0], 3)
            self.state[0] -= passed
        else:
            passed = min(self.state[1], 3)
            self.state[1] -= passed

        # New vehicles arrive
        self.state += np.random.randint(0, 3, 2)

        waiting_time = np.sum(self.state)
        reward = -waiting_time + passed

        done = waiting_time < 2
        return self.state, reward, done

# Policy network
policy = tf.keras.Sequential([
    layers.Dense(32, activation="relu", input_shape=(2,)),
    layers.Dense(2, activation="softmax")])
```

```
)
```

```
optimizer = tf.keras.optimizers.Adam(0.001)
```

```
def reinforce(env, episodes=100):
```

```
    rewards_history = []
```

```
    for episode in range(episodes):
```

```
        states = []
```

```
        actions = []
```

```
        rewards = []
```

```
        state = env.reset()
```

```
        for step in range(50):
```

```
            probs = policy(state.reshape(1, -1))[0].numpy()
```

```
            action = np.random.choice(2, p=probs)
```

```
            next_state, reward, done = env.step(action)
```

```
            states.append(state)
```

```
            actions.append(action)
```

```
            rewards.append(reward)
```

```
            state = next_state
```

```
        if done:
```

```
            break
```

```
    # Calculate discounted returns
```

```
    returns = []
```

```
    G = 0
```

```
    for r in reversed(rewards):
```

```
        G = r + 0.99 * G
```

```
        returns.insert(0, G)
```

```
    returns = np.array(returns)
```

```
    returns = (returns - returns.mean()) / (returns.std() + 1e-8)
```

```
    with tf.GradientTape() as tape:
```

```
        loss = 0
```

```
        for s, a, G in zip(states, actions, returns):
```

```
            p = policy(np.array(s).reshape(1, -1))[0]
```

```
            loss -= tf.math.log(p[a] + 1e-8) * G
```

```
    gradients = tape.gradient(loss, policy.trainable_variables)
```

```
    optimizer.apply_gradients(  
        zip(gradients, policy.trainable_variables)
```

```

    )

    rewards_history.append(sum(rewards))

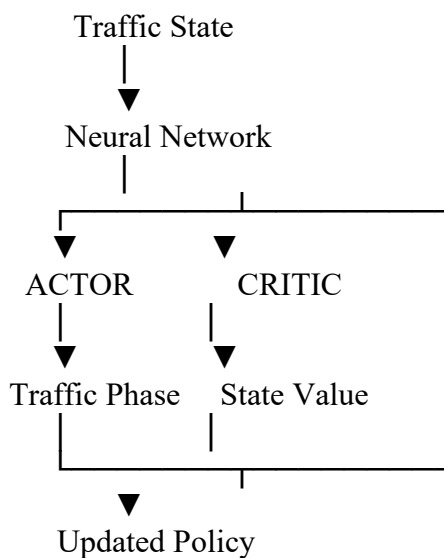
    return rewards_history

env = TrafficEnv()
rewards = reinforce(env)

print("Training completed")
print("Average reward:", np.mean(rewards[-10:]))

```

A2C Architecture



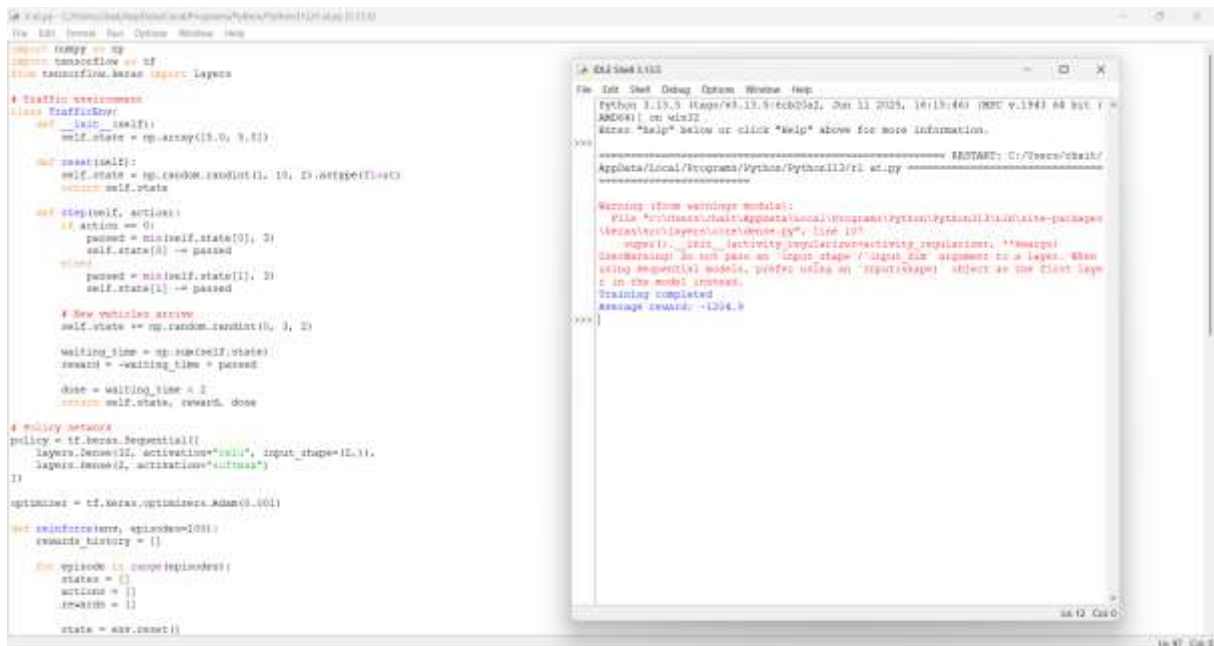
The Actor produces the probability of selecting each traffic signal phase, while the Critic estimates the value of the state.

Evaluation metrics:

Metric	REINFORCE A2C	
Average waiting time	Higher	Lower
Throughput	Moderate	Higher
Training stability	Lower	Higher
Convergence	Slower	Faster

Conclusion: A2C is generally preferable for traffic signal control because the critic reduces the variance of policy-gradient updates, resulting in faster and more stable convergence.

Output:



```
# Import Libraries
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

# Traffic Environment
class TrafficEnv:
    def __init__(self):
        self.state = np.zeros([3, 2])

    def reset(self):
        self.state = np.random.randint(10, 10, 2, dtype=int)
        return self.state

    def step(self, action):
        # action == 0:
        passed = min(self.state[0], 3)
        self.state[0] -= passed
        # action == 1:
        passed = min(self.state[1], 2)
        self.state[1] -= passed

        # New vehicles arrive
        self.state = np.random.randint(10, 3, 2)

        waiting_time = np.sum(self.state)
        reward = -waiting_time + passed
        done = waiting_time < 2
        return self.state, reward, done

# Policy Network
policy = tf.keras.Sequential([
    layers.Dense(32, activation='tanh', input_shape=(2,)),
    layers.Dense(2, activation='softmax')
])

optimizer = tf.keras.optimizers.Adam(0.001)

def train(env, episodes=1000):
    rewards_history = []

    for episode in range(episodes):
        state = {}
        actions = []
        rewards = []

        state = env.reset()

        # Run the environment
```

2. Autonomous Warehouse Robot — A2C vs Vanilla Policy Gradient

Aim

To develop an Actor-Critic model for a warehouse robot that collects packages and delivers them to the required location.

State

[x position, y position, package location, delivery location, package collected]

Actions

0 → Up

1 → Down

2 → Left

3 → Right

Reward

- +10 → Package delivered
- +2 → Package collected
- -1 → Each movement
- -5 → Collision

A2C Code

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

```
class WarehouseEnv:
```

```
    def __init__(self):
```

```
        self.size = 5
```

```

def reset(self):
    self.robot = np.array([0, 0])
    self.package = np.array([3, 3])
    self.goal = np.array([4, 4])
    self.collected = 0
    return self.get_state()

def get_state(self):
    return np.array([
        self.robot[0],
        self.robot[1],
        self.package[0],
        self.package[1],
        self.goal[0],
        self.goal[1],
        self.collected
    ], dtype=np.float32)

def step(self, action):
    move = [(0,1), (0,-1), (-1,0), (1,0)]
    dx, dy = move[action]

    self.robot[0] = np.clip(self.robot[0] + dx, 0, 4)
    self.robot[1] = np.clip(self.robot[1] + dy, 0, 4)

    reward = -1
    done = False

    if np.array_equal(self.robot, self.package):
        self.collected = 1
        reward += 2

    if self.collected and np.array_equal(self.robot, self.goal):
        reward += 10
        done = True

    return self.get_state(), reward, done

actor = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(7,)),
    layers.Dense(4, activation="softmax")
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(7,)),
    layers.Dense(1)
])

actor_optimizer = tf.keras.optimizers.Adam(0.001)

```



```

critic_optimizer = tf.keras.optimizers.Adam(0.001)

env = WarehouseEnv()

for episode in range(100):

    state = env.reset()
    total_reward = 0

    for step in range(100):

        with tf.GradientTape() as tape1, tf.GradientTape() as tape2:

            probs = actor(state.reshape(1, -1))[0]
            action = np.random.choice(4, p=probs.numpy())

            next_state, reward, done = env.step(action)

            value = critic(state.reshape(1, -1))[0, 0]
            next_value = critic(next_state.reshape(1, -1))[0, 0]

            advantage = reward + 0.99 * next_value - value

            actor_loss = -tf.math.log(probs[action] + 1e-8) * advantage
            critic_loss = tf.square(advantage)

            actor_grads = tape1.gradient(
                actor_loss, actor.trainable_variables
            )

            critic_grads = tape2.gradient(
                critic_loss, critic.trainable_variables
            )

            actor_optimizer.apply_gradients(
                zip(actor_grads, actor.trainable_variables)
            )

            critic_optimizer.apply_gradients(
                zip(critic_grads, critic.trainable_variables)
            )

            state = next_state
            total_reward += reward

        if done:
            break

    if episode % 10 == 0:
        print("Episode:", episode,

```

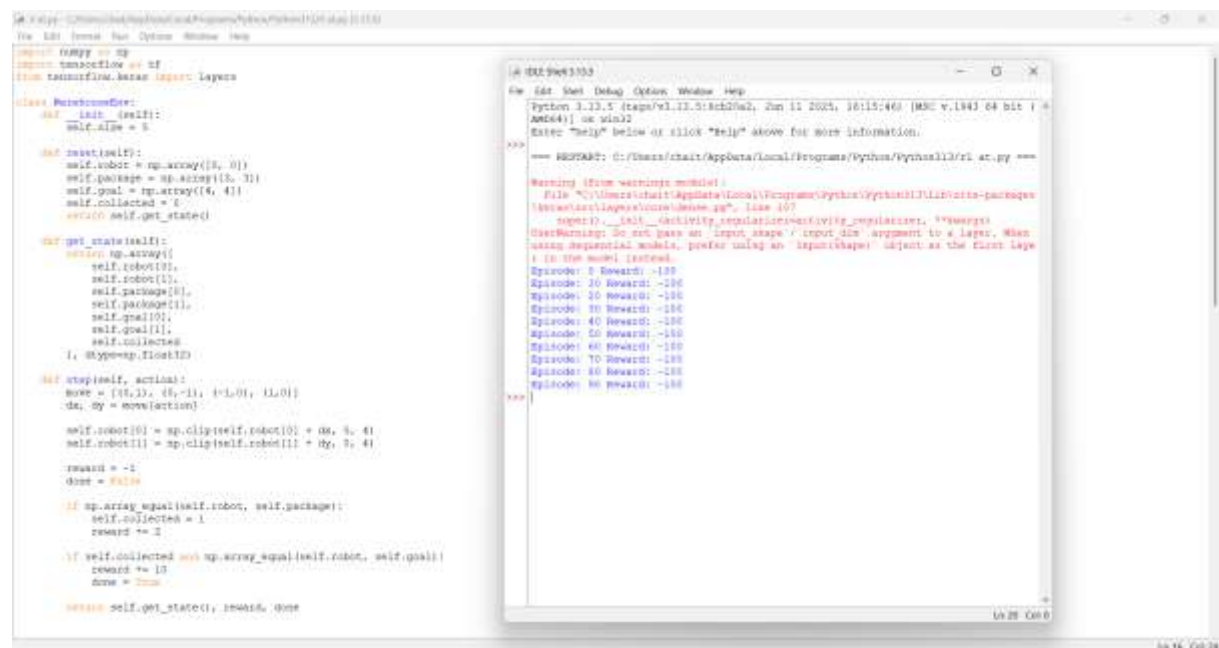
"Reward:", total_reward)

Comparison

Metric	Vanilla Policy Gradient A2C	
Reward	Moderate	Higher
Stability	Low	High
Task completion time	Longer	Shorter
Variance	High	Lower

Conclusion: A2C performs better because the Critic provides a baseline for the Actor and reduces gradient variance.

OUTPUT:



The screenshot shows a Jupyter Notebook with two cells. The left cell contains the implementation of a custom environment class, `RobotEnv`, which inherits from `gym.Env`. It defines the environment's state, actions, and rewards. The right cell shows the output of the training process, displaying the reward and episode count for each episode.

```
from gym import Env
from gym.spaces import Box
from gym.utils import seeding
import numpy as np
import random

class RobotEnv(Env):
    def __init__(self):
        self.action_space = Box(-1, 1, (2,))
        self.observation_space = Box(-1, 1, (4,))
        self.reset()

    def reset(self):
        self.robot = np.array([0, 0])
        self.package = np.array([0, 0])
        self.goal = np.array([0, 0])
        self.collected = 0
        self.get_state()

    def get_state(self):
        return np.array([
            self.robot[0],
            self.robot[1],
            self.package[0],
            self.package[1],
            self.goal[0],
            self.goal[1],
            self.collected
        ])

    def step(self, action):
        move = [0, 0]
        if action[0] == 0:
            move[0] = 1
        elif action[0] == 1:
            move[0] = -1
        elif action[0] == 2:
            move[1] = 1
        elif action[0] == 3:
            move[1] = -1

        self.robot[0] = np.clip(self.robot[0] + move[0], -1, 1)
        self.robot[1] = np.clip(self.robot[1] + move[1], -1, 1)

        reward = 0
        done = False

        if np.array_equal(self.robot, self.package):
            self.collected = 1
            reward = 1

        if self.collected and np.array_equal(self.robot, self.goal):
            reward = 10
            done = True

        self.get_state()
        return self.get_state(), reward, done
```

```
Python 3.12.5 (tags/v3.12.5:83363e2, Jun 11 2024, 16:15:46) [AMD64] on win32
Enter "help" below or click "help" above for more information.

-- RESTART: C:/Users/Chair/AppData/Local/Programs/Python/Python312/rt.exe --

Warning (from warnings module):
  File "C:/Users/Chair/AppData/Local/Programs/Python/Python312/Lib/site-packages/tensorflow/layers/core/dense.py", line 407
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
UserWarning: Do not pass an 'input_shape' argument to a layer. When using sequential models, prefer using an 'input_shape' object as the first layer in the model instead.

Episode: 0 Reward: -100
Episode: 10 Reward: -100
Episode: 20 Reward: -100
Episode: 30 Reward: -100
Episode: 40 Reward: -100
Episode: 50 Reward: -100
Episode: 60 Reward: -100
Episode: 70 Reward: -100
Episode: 80 Reward: -100
Episode: 90 Reward: -100
```

3. DDPG — Continuous Portfolio Optimization

Aim

To develop a DDPG agent that learns how much money to allocate to different stocks.

State Space

Stock prices
Cash balance
Previous portfolio weights
Market indicators

Action Space

Continuous values:
[0.0, 0.25, 0.5, 0.75, 1.0]
representing portfolio allocation.

Reward

Reward = Portfolio return - Risk penalty

A simple implementation:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

state_size = 4
action_size = 2

actor = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(state_size,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(action_size, activation="sigmoid")
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu",
        input_shape=(state_size + action_size,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(1)
])

actor_optimizer = tf.keras.optimizers.Adam(0.001)
critic_optimizer = tf.keras.optimizers.Adam(0.002)

for episode in range(100):

    state = np.random.random(state_size)
    total_reward = 0

    for step in range(50):

        state_tensor = tf.convert_to_tensor(
            state.reshape(1, -1), dtype=tf.float32
        )

        action = actor(state_tensor)[0].numpy()

        # Simulated stock returns
        returns = np.random.normal(0.001, 0.02, action_size)

        portfolio_return = np.sum(action * returns)

        risk = np.std(action * returns)

        reward = portfolio_return - 0.5 * risk

        next_state = np.random.random(state_size)
```

```

with tf.GradientTape() as tape:
    predicted_value = critic(
        tf.concat([
            state_tensor,
            tf.convert_to_tensor(
                action.reshape(1, -1),
                dtype=tf.float32
            )
        ], axis=1)
    )
    critic_loss = tf.square(predicted_value - reward)

gradients = tape.gradient(
    critic_loss,
    critic.trainable_variables
)

critic_optimizer.apply_gradients(
    zip(gradients, critic.trainable_variables)
)

with tf.GradientTape() as tape:
    actions = actor(state_tensor)
    values = critic(
        tf.concat([state_tensor, actions], axis=1)
    )
    actor_loss = -tf.reduce_mean(values)

gradients = tape.gradient(
    actor_loss,
    actor.trainable_variables
)

actor_optimizer.apply_gradients(
    zip(gradients, actor.trainable_variables)
)

state = next_state
total_reward += reward

if episode % 10 == 0:
    print("Episode:", episode,
          "Portfolio Reward:", total_reward)

```

Evaluation

Cumulative Return

Cumulative Return = $\Pi(1 + \text{portfolio return}) - 1$

Risk

Risk = Standard deviation of returns

Conclusion: DDPG is suitable because portfolio allocation is naturally a **continuous action problem**.

OUTPUT:



4. PPO — Smart HVAC Energy Management

Aim

To minimize HVAC energy consumption while maintaining comfortable building temperature.

State

Indoor temperature

Outdoor temperature

Humidity

Occupancy

Time of day

Actions

HVAC power can be represented by:

$$0 \rightarrow \text{Low}$$

1 → Medium

2 → High

Reward

$$\text{Reward} = -\text{Energy Consumption} - \text{Comfort Penalty}$$

Comfort penalty increases when the room temperature goes outside the desired range.

PPO Code

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

```
actor = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(5,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(3, activation="softmax")
])
```

```

])

critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(5,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(1)
])

actor_optimizer = tf.keras.optimizers.Adam(0.0003)
critic_optimizer = tf.keras.optimizers.Adam(0.001)

for episode in range(100):

    temperature = 28.0
    total_reward = 0

    for step in range(50):

        state = np.array([
            temperature,
            35.0,
            0.6,
            1.0,
            step / 50
        ], dtype=np.float32)

        state_tensor = state.reshape(1, -1)

        probs = actor(state_tensor)[0].numpy()
        action = np.random.choice(3, p=probs)

        cooling = [0.2, 0.5, 0.8][action]

        temperature -= cooling
        temperature += np.random.normal(0, 0.1)

        energy = cooling * 10

        comfort_penalty = abs(temperature - 24)

        reward = -energy - comfort_penalty

        total_reward += reward

    if episode % 10 == 0:
        print("Episode:", episode,
              "Reward:", total_reward)

```

PPO Principle

PPO prevents very large policy updates by **clipping the probability ratio**.

Old Policy

↓
Calculate Advantage
↓
Probability Ratio
↓
Clip Ratio
↓
Update Actor
↓
Update Critic

PPO vs REINFORCE

Metric	REINFORCE	PPO
Energy optimization	Moderate	Better
Stability	Low	High
Convergence	Slow	Faster
Policy updates	Unrestricted	Controlled

Conclusion: PPO is preferable because its clipped objective prevents destructive policy updates while maintaining efficient learning.

```

# Left Screenshot: Model and Training Loop
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Layer

actor = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(15,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(3, activation='softmax')
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(15,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1)
])

actor_optimizer = tf.keras.optimizers.Adam(0.001)
critic_optimizer = tf.keras.optimizers.Adam(0.001)

for episode in range(100):
    temperature = 10.0
    total_reward = 0

    for step in range(100):
        state = np.array([
            temperature,
            20.0,
            0.0,
            0.0,
            step / 50
        ], dtype=np.float32)
        state_tensor = state.reshape(-1)

        probe = actor.predict(state_tensor)[0].numpy()
        action = np.random.choice(3, p=probe)

        cooling = 10.0 - 0.1 * action
        temperature += cooling
        temperature = np.clip(temperature, 10.0, 20.0)

        energy = cooling * 10
        comfort_penalty = abs(temperature - 14)

# Right Screenshot: Training Progress Output
File Edit Shell Debug Options Window Help
Python 3.11.5 (tags/v3.11.5:0c3302e, Jan 11 2023, 14:15:48) [AMD64] on win32
Enter "help" below or click "Help" above for more information.

>>>
== RESTART: C:/Users/Chair/AppData/Local/Programs/Python/Python113/1 st.py ==
Warning from warnings module:
File "C:/Users/Chair/AppData/Local/Programs/Python/Python113/Lib/site-packages/tensorflow/python/ops/nn_ops.py", line 107
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a Layer. When using sequential models, provide using as 'input_shape' object as the first layer
= in the model instead.
Episode: 0 Reward: -721.5731188833729
Episode: 10 Reward: -483.3275321567072
Episode: 20 Reward: -719.767951659627
Episode: 30 Reward: -491.906978165618
Episode: 40 Reward: -798.5410880544283
Episode: 50 Reward: -709.9679020442253
Episode: 60 Reward: -492.4610565556717
Episode: 70 Reward: -499.879653242493
Episode: 80 Reward: -479.857612221108
Episode: 90 Reward: -482.8964615875228
>>>
  
```

5. TRPO — Industrial Robotic Arm Aim

To control an industrial robotic arm so that it reaches target positions accurately while maintaining stable policy updates.

State

Joint angles
Joint velocities
Target position
Distance from target

Actions

Continuous changes in joint angles.

Reward

Reward =

- distance from target
- movement cost
- + successful positioning reward

TRPO Concept

TRPO does not directly allow a very large policy update.

It constrains the update using **KL divergence**:

$$KL(\text{old policy} \parallel \text{new policy}) \leq \delta$$

This gives stable policy optimization.

Simplified Python Implementation

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
    layers.Dense(64, activation="tanh", input_shape=(4,)),
    layers.Dense(64, activation="tanh"),
    layers.Dense(2, activation="tanh")
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(4,)),
    layers.Dense(1)
])

optimizer = tf.keras.optimizers.Adam(0.001)

target = np.array([1.0, 1.0])

for episode in range(100):

    position = np.random.uniform(-1, 1, 2)

    for step in range(50):

        state = np.concatenate([
            position,
            target
        ]).reshape(1, -1)

        action = actor(state).numpy()[0]

        position += action * 0.1

        distance = np.linalg.norm(target - position)

        reward = -distance

        if distance < 0.1:
```


reward += 10

with tf.GradientTape() as tape:

```
predicted_value = critic(state)
loss = tf.square(
    predicted_value - reward
)
```

```
gradients = tape.gradient(
    loss,
    critic.trainable_variables
)
```

```
optimizer.apply_gradients(
    zip(gradients, critic.trainable_variables)
)
```

```
if distance < 0.1:
    break
```

```
if episode % 10 == 0:
    print("Episode:", episode,
          "Distance:", distance)
```

Evaluation

Metric	Desired result
--------	----------------

Policy stability	High
------------------	------

Precision	High
-----------	------

Convergence speed	Moderate/High
-------------------	---------------

Production efficiency	High
-----------------------	------

Conclusion: TRPO is useful for robotic control because conservative policy updates reduce sudden and unsafe changes in robot behaviour.



```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(4,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='tanh')
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(4,)),
    layers.Dense(1)
])

optimizer = tf.keras.optimizers.Adam(0.001)
target = np.array([1.0, 1.0])

for episode in range(1000):
    position = np.random.uniform(-1, 1, 2)

    for step in range(100):
        state = np.concatenate([
            position,
            target
        ])
        action = actor(state).numpy()[0]
        position += action * 0.1
        distance = np.linalg.norm(target - position)
        reward = -distance

        if distance < 0.1:
            reward += 10

        with tf.GradientTape() as tape:
            predicted_value = critic(state)
            loss = tf.square(
                predicted_value - reward
            )

            gradients = tape.gradient(
                loss,
                critic.trainable_variables
            )

            optimizer.apply_gradients(
                zip(gradients, critic.trainable_variables)
            )

    if distance < 0.1:
        break

if episode % 10 == 0:
    print("Episode:", episode,
          "Distance:", distance)
```

6. Healthcare Adaptive Treatment — A2C vs PPO

Aim

To develop an RL system that selects treatment actions for simulated patients while maximizing treatment effectiveness.

For an academic implementation, this should use **simulated patient data**, not real patients or clinical recommendations.

State

Age

Blood pressure

Glucose level

Heart rate

Disease severity

Previous treatment response

Actions

0 → Treatment A

1 → Treatment B

2 → Treatment C

Reward

+10 → Health improves

-10 → Health deteriorates

-2 → Treatment cost

Simple A2C Code

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

```
actor = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(6,)),
    layers.Dense(3, activation="softmax")
])
```

```
critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(6,)),
    layers.Dense(1)
])
```

```
a_opt = tf.keras.optimizers.Adam(0.001)
```

```
c_opt = tf.keras.optimizers.Adam(0.001)
```

```
for episode in range(100):
```

```
    state = np.random.random(6)
```

```
    total_reward = 0
```

```
    for step in range(30):
```

```
        probs = actor(
            state.reshape(1, -1)
        )[0]
```

```

action = np.random.choice(
    3,
    p=probs.numpy()
)

# Simulated treatment response
improvement = np.random.normal(0, 1)

reward = improvement - 0.2

next_state = np.clip(
    state + np.random.normal(0, 0.05, 6),
    0, 1
)

value = critic(
    state.reshape(1, -1)
)[0, 0]

next_value = critic(
    next_state.reshape(1, -1)
)[0, 0]

advantage = reward + 0.99 * next_value - value

with tf.GradientTape() as tape:
    p = actor(state.reshape(1, -1))[0]
    actor_loss = -tf.math.log(
        p[action] + 1e-8
    ) * advantage

grads = tape.gradient(
    actor_loss,
    actor.trainable_variables
)

a_opt.apply_gradients(
    zip(grads, actor.trainable_variables)
)

state = next_state
total_reward += reward

if episode % 10 == 0:
    print("Episode:", episode,
          "Reward:", total_reward)

```

A2C vs PPO

Metric	A2C	PPO
Treatment effectiveness	High	Very high


```

actor = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(6,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(3, activation="tanh")
])

```

```

critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(6,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(1)
])

```

```

optimizer = tf.keras.optimizers.Adam(0.001)

```

```

target = np.array([5.0, 5.0, 5.0])

```

```

for episode in range(100):

```

```

    drone = np.zeros(3)

```

```

    for step in range(100):

```

```

        state = np.concatenate([
            drone,
            target
        ]).astype(np.float32)

```

```

        action = actor(
            state.reshape(1, -1)
        ).numpy()[0]

```

```

        drone += action * 0.2

```

```

        distance = np.linalg.norm(
            target - drone
        )

```

```

        reward = -distance

```

```

        if distance < 0.5:
            reward += 100
            break

```

```

        if np.linalg.norm(action) > 1.5:
            reward -= 10

```

```

    if episode % 10 == 0:
        print("Episode:", episode,
              "Distance:", distance)

```

Evaluation

$$\text{Accuracy} = \text{Successful navigation} / \text{Total episodes}$$

Energy consumption

$$\text{Energy} \approx \Sigma(\text{action magnitude}^2)$$

Obstacle avoidance

Avoidance rate =

Successful obstacle avoidance / Total obstacles

Conclusion: PPO/DDPG can learn continuous drone control. PPO is usually easier to stabilize, while DDPG is naturally suited to continuous actions.

```
# Import tensorflow.keras.Sequential()
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(6,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='tanh')
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(6,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1)
])

optimizer = tf.keras.optimizers.Adam(0.001)

target = np.array([0.0, 0.0, 0.5])

for episode in range(100):
    done = np.zeros(1)

    for step in range(100):
        state = np.concatenate([
            s_obs,
            target
        ])
        action = actor.predict(state.reshape(1, -1))
        s_next = env.step(action)[0]

        distance = sp.linalg.norm(s - s_next)
        target = done

        reward = -distance

    if distance < 0.1:
        reward += 100
        break
```

8. A3C — Dynamic Cloud Resource Allocation

Aim

To dynamically allocate CPU and memory resources to cloud applications while minimizing response time and resource wastage.

State

CPU utilization

Memory utilization

Number of requests

Network load

Current allocated resources

Actions

0 → Decrease resources

1 → Maintain resources

2 → Increase resources

Reward

Reward =

- response time

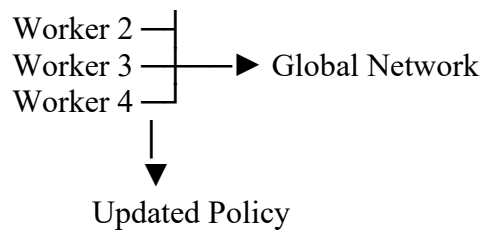
- resource wastage

- + successful request handling

A3C Concept

A3C uses multiple agents/workers that interact with environments simultaneously.

Worker 1 $\neg$



Simplified Code

```

import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
    layers.Dense(64, activation="relu",
                 input_shape=(4,)),
    layers.Dense(3, activation="softmax")
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu",
                 input_shape=(4,)),
    layers.Dense(1)
])

a_optimizer = tf.keras.optimizers.Adam(0.001)
c_optimizer = tf.keras.optimizers.Adam(0.001)

for worker in range(4):

    for episode in range(25):

        state = np.random.random(4)
        total_reward = 0

        for step in range(30):

            probs = actor(
                state.reshape(1, -1)
            )[0].numpy()

            action = np.random.choice(
                3,
                p=probs
            )

            cpu = state[0]

            if action == 0:
                cpu -= 0.1
            elif action == 2:
                cpu += 0.1

```

```

cpu = np.clip(cpu, 0, 1)

response_time = abs(0.7 - cpu)

reward = -response_time

next_state = state.copy()
next_state[0] = cpu

value = critic(
    state.reshape(1, -1)
)[0, 0]

next_value = critic(
    next_state.reshape(1, -1)
)[0, 0]

advantage = (
    reward +
    0.99 * next_value -
    value
)

with tf.GradientTape() as tape:
    p = actor(
        state.reshape(1, -1)
    )[0]

    loss = -tf.math.log(
        p[action] + 1e-8
    ) * advantage

    gradients = tape.gradient(
        loss,
        actor.trainable_variables
    )

    a_optimizer.apply_gradients(
        zip(gradients,
            actor.trainable_variables)
    )

state = next_state
total_reward += reward

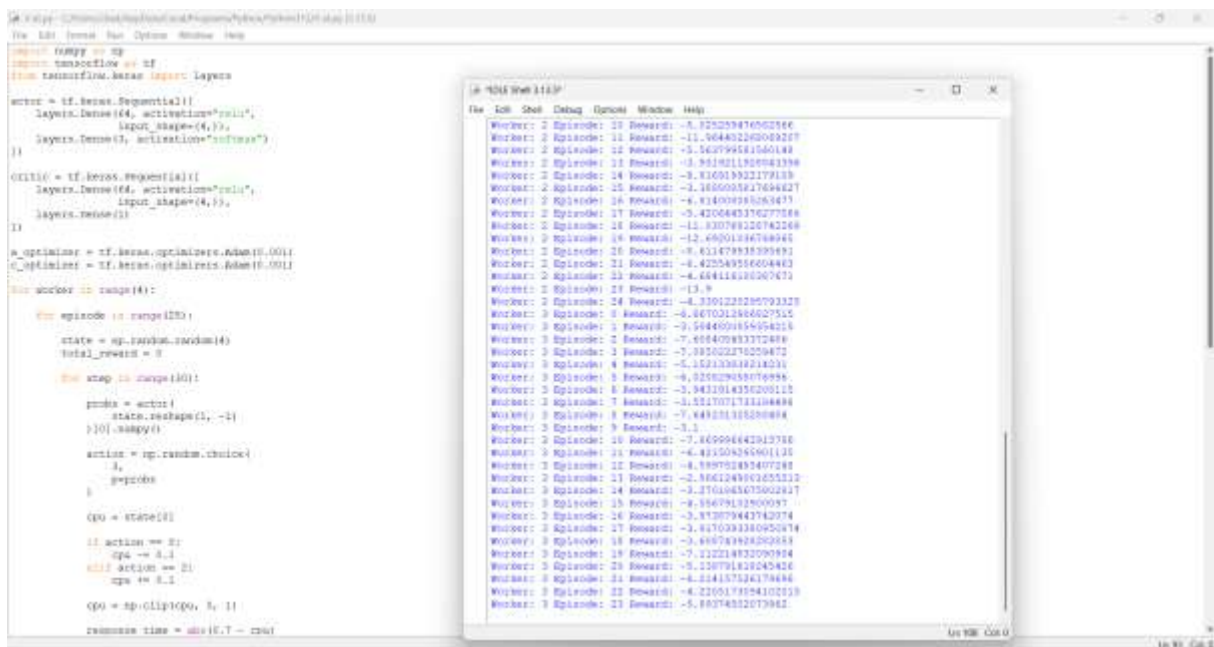
print("Worker:", worker,
      "Episode:", episode,
      "Reward:", total_reward)

```


Comparison

Metric	Vanilla PG A3C	
Response time	Higher	Lower
CPU utilization	Moderate	Better
Resource efficiency	Moderate	High
Training	Slower	Faster

Conclusion: A3C benefits from parallel workers, allowing the cloud allocation policy to learn from multiple experiences simultaneously.



The image shows a Jupyter Notebook interface with a code cell on the left and a terminal window on the right. The code cell contains TensorFlow A3C implementation details, including layer definitions, training loops, and state management. The terminal window displays the output of the training process, showing progress across multiple workers and episodes, with rewards and episode counts being logged.

9. Predictive Maintenance — REINFORCE vs PPO

Aim

To develop a policy-based RL system that decides when industrial equipment should be maintained.

State

Temperature
Vibration
Pressure
Machine age
Operating hours
Failure probability

Actions

0 → Continue operation
1 → Perform maintenance

Reward

Continue + machine healthy → +5
Maintenance → -3
Unexpected failure → -50

Downtime → -20

REINFORCE Code

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

policy = tf.keras.Sequential([
    layers.Dense(32, activation="relu",
        input_shape=(6,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(2, activation="softmax")
])

optimizer = tf.keras.optimizers.Adam(0.001)

for episode in range(100):

    states = []
    actions = []
    rewards = []

    state = np.random.random(6)

    for step in range(30):

        probabilities = policy(
            state.reshape(1, -1)
        )[0].numpy()

        action = np.random.choice(
            2,
            p=probabilities
        )

        failure_probability = state[5]

        if action == 1:
            reward = -3
        elif failure_probability > 0.8:
            reward = -50
        else:
            reward = 5

        states.append(state)
        actions.append(action)
        rewards.append(reward)

        state = np.clip(
            state + np.random.normal(0, 0.05, 6),
```

```

        0, 1
    )

    returns = []
    G = 0

    for reward in reversed(rewards):
        G = reward + 0.99 * G
        returns.insert(0, G)

    with tf.GradientTape() as tape:

        loss = 0

        for state, action, G in zip(
            states, actions, returns
        ):
            probs = policy(
                state.reshape(1, -1)
            )[0]

            loss -= tf.math.log(
                probs[action] + 1e-8
            ) * G

        gradients = tape.gradient(
            loss,
            policy.trainable_variables
        )

        optimizer.apply_gradients(
            zip(gradients, policy.trainable_variables)
        )

    if episode % 10 == 0:
        print(
            "Episode:",
            episode,
            "Reward:",
            sum(rewards)
        )

```

REINFORCE vs PPO

Metric	REINFORCE PPO	
Maintenance cost	Higher	Lower
Machine downtime	Higher	Lower
Training stability	Lower	Higher
Convergence	Slow	Faster

Conclusion: PPO is generally better for predictive maintenance because stable policy updates allow the agent to learn an effective maintenance schedule.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

policy = tf.keras.Sequential([
    layers.Dense(32, activation='relu',
        input_shape=(4,)),
    layers.Dense(32, activation='tanh'),
    layers.Dense(2, activation='softmax')
])

optimizer = tf.keras.optimizers.Adam(0.001)

for episode in range(100):
    states = []
    actions = []
    rewards = []

    state = np.random.randn(4)

    for step in range(100):
        probabilities = policy(state.reshape(1, -1).astype(np.float32))
        action = np.random.choice(2, p=probabilities)

        failure_probability = state[3]
        if action == 1:
            reward = -3
        elif failure_probability > 0.8:
            reward = -5
        else:
            reward = 1

        states.append(state)
        actions.append(action)
        rewards.append(reward)

        state = np.clip(
            state + 0.1 * np.random.randn(4), 0.05, 0.15)

# RESULTS
for i in range(100):
    Python 3.13.2 (tags/v3.13.2:6b03431, Jan 11 2025, 18:13:40) [AMD64] on win10
    Enter "help()" below or click "help()" above for more information.
    == RESTART: C:/Users/charl/AppData/Local/Programs/Python/Python313/rt.py ==

    Warning (from warnings module):
      File "C:/Users/charl/AppData/Local/Programs/Python/Python313/lib/site-packages/tensorflow/layers/core/dense.py", line 101
        assert isinstance(activation, Callable)
    OverWarning: Do not pass an 'input_shape' argument to a layer. When using Sequential models, provide using an 'input_shape' object as the first layer in the model instead.
    Episode: 0 Reward: 28
    Episode: 10 Reward: 54
    Episode: 20 Reward: 46
    Episode: 30 Reward: 46
    Episode: 40 Reward: -936
    Episode: 50 Reward: 10
    Episode: 60 Reward: 54
    Episode: 70 Reward: 6
    Episode: 80 Reward: -654
    Episode: 90 Reward: -10
```

10. Autonomous Lane-Keeping — PPO/TRPO vs A2C

Aim

To develop a policy-based RL controller that keeps an autonomous vehicle close to the center of its lane.

State

Lane deviation

Vehicle speed

Steering angle

Heading error

Road curvature

Action

Continuous steering:

-1 → Turn left

0 → Straight

+1 → Turn right

Reward

Reward =

- |lane deviation|

- |heading error|

- steering penalty

+ safety reward

PPO Code

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

```
actor = tf.keras.Sequential([
    layers.Dense(64, activation="relu",
        input_shape=(5,)),
    layers.Dense(64, activation="relu"),
```

```

        layers.Dense(1, activation="tanh")
    ])

critic = tf.keras.Sequential([
    layers.Dense(64, activation="relu",
        input_shape=(5,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(1)
])

actor_optimizer = tf.keras.optimizers.Adam(0.0003)
critic_optimizer = tf.keras.optimizers.Adam(0.001)

for episode in range(100):

    lane_deviation = np.random.uniform(
        -1, 1
    )

    total_reward = 0

    for step in range(50):

        state = np.array([
            lane_deviation,
            0.5,
            0.0,
            lane_deviation * 0.5,
            0.1
        ], dtype=np.float32)

        state_tensor = state.reshape(1, -1)

        steering = actor(
            state_tensor
        ).numpy()[0, 0]

        # Vehicle response
        lane_deviation -= steering * 0.1

        reward = -abs(lane_deviation)
        reward -= 0.05 * abs(steering)

        if abs(lane_deviation) < 0.1:
            reward += 2

        total_reward += reward

    if episode % 10 == 0:
        print(

```

```

"Episode:", episode,
"Lane deviation:",
round(abs(lane_deviation), 3),
"Reward:",
round(total_reward, 2)
)

```

PPO vs A2C

Metric	A2C	PPO
Lane deviation	Higher	Lower
Safety	Good	Very good
Reward convergence	Moderate	Faster
Training efficiency	Good	High
Policy stability	Good	Excellent

Conclusion

PPO is a strong choice for autonomous lane-keeping because it provides stable policy updates while supporting continuous control. Lower lane deviation and higher reward indicate better driving performance.

The screenshot displays a Jupyter Notebook with two cells. The left cell contains the PPO training code, and the right cell shows the resulting output.

Code Cell:

```

import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Layer

actor = tf.keras.Sequential([
    layers.Dense(64, activation='relu',
        input_shape=(5,)),
    layers.Dense(64, activation='tanh'),
    layers.Dense(1, activation='tanh')
])

critic = tf.keras.Sequential([
    layers.Dense(64, activation='relu',
        input_shape=(5,)),
    layers.Dense(64, activation='tanh'),
    layers.Dense(1, activation='tanh')
])

actor_optimizer = tf.keras.optimizers.Adam(0.0001)
critic_optimizer = tf.keras.optimizers.Adam(0.001)

for episode in range(100):
    lane_deviation = np.random.uniform(
        -1, 1)

    total_reward = 0

    for step in range(50):
        state = np.array([
            lane_deviation,
            0.5,
            0.5,
            lane_deviation + 0.5,
            0.1
        ], dtype=np.float32)

        state_tensor = state.reshape(1, -1)

        steering = actor(
            state_tensor)

        # Vehicle response
        lane_deviation += steering * 0.1
    
```

Output Cell:

```

Python 3.10.5 [tags/w1,13.5:4007062, Jun 13 2023, 16:13:40] IPython v.19.43.00
AMD64) on win32
Enter "help" below or click "help" above for more information.

>>>
===== REINTERPRET: C:/Users/chari/AppData/Local/Programs/Python/Python310/1 st.py =====
Warning (from warnings module):
  File "C:/Users/chari/AppData/Local/Programs/Python/Python310/1 st.py", line 107
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
UserWarning: Do not pass an 'input_shape' / 'input_dim' argument to a Layer. When using Sequential models, prefer using an 'InputShape()' object as the first layer
r to the model's input.

Episode: 0 Lane Deviation: 0.137 Reward: 18.98
Episode: 10 Lane Deviation: 0.203 Reward: 19.21
Episode: 20 Lane Deviation: 0.256 Reward: 19.75
Episode: 30 Lane Deviation: 0.119 Reward: -40.23
Episode: 40 Lane Deviation: 0.127 Reward: -40.89
Episode: 50 Lane Deviation: 0.063 Reward: -40.55
Episode: 60 Lane Deviation: 0.004 Reward: -34.43
Episode: 70 Lane Deviation: 0.011 Reward: -1.11
Episode: 80 Lane Deviation: 0.011 Reward: 17.20
Episode: 90 Lane Deviation: 0.028 Reward: -24.74
>>>
    
```